

Tutorial 4: More about Controllers (30 Minutes)

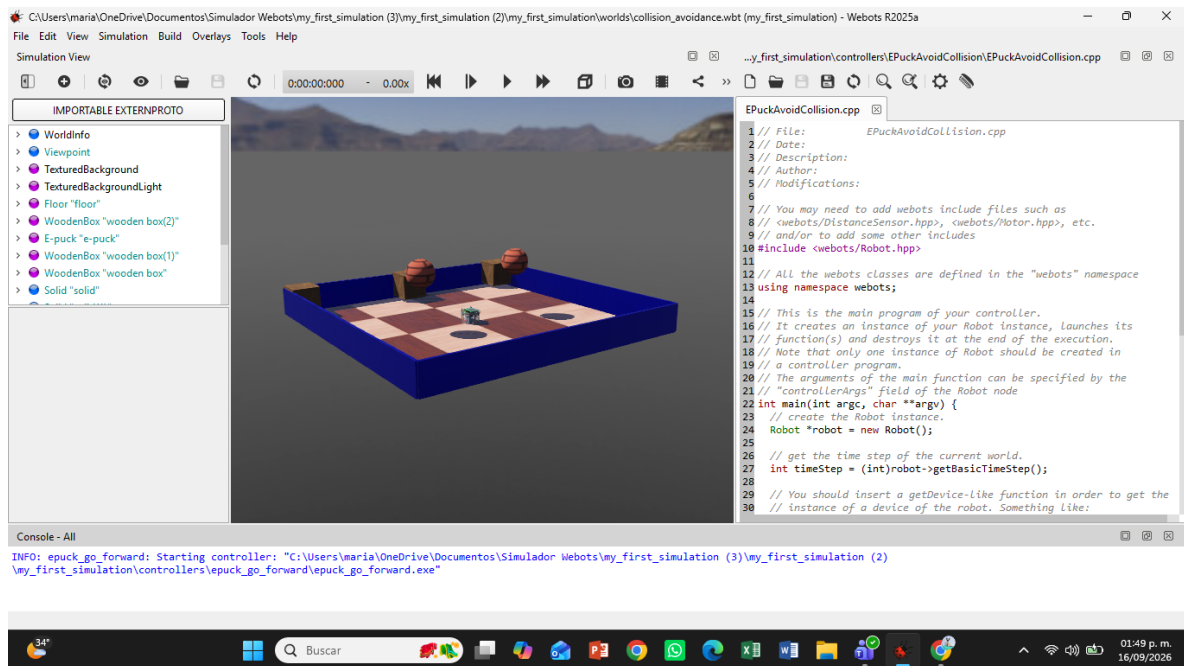
Familiarizarse con la programación de controladores de robots en Webots, vinculando los nodos del árbol de escena con la API de programación, el ciclo de vida de un controlador (inicialización y limpieza), la lectura de sensores, el control de actuadores y la implementación de un algoritmo básico de retroalimentación para evitar obstáculos.

Objetivo de la Práctica: Diseñar y programar un controlador simple en lenguaje C (u otro lenguaje compatible) para el robot *e-puck*, que permita leer los valores de sus 8 sensores infrarrojos de distancia distribuidos alrededor de su torreta y accionar sus dos motores de ruedas de manera coordinada para evitar colisiones con los obstáculos (cajas y paredes) creados en las prácticas anteriores.

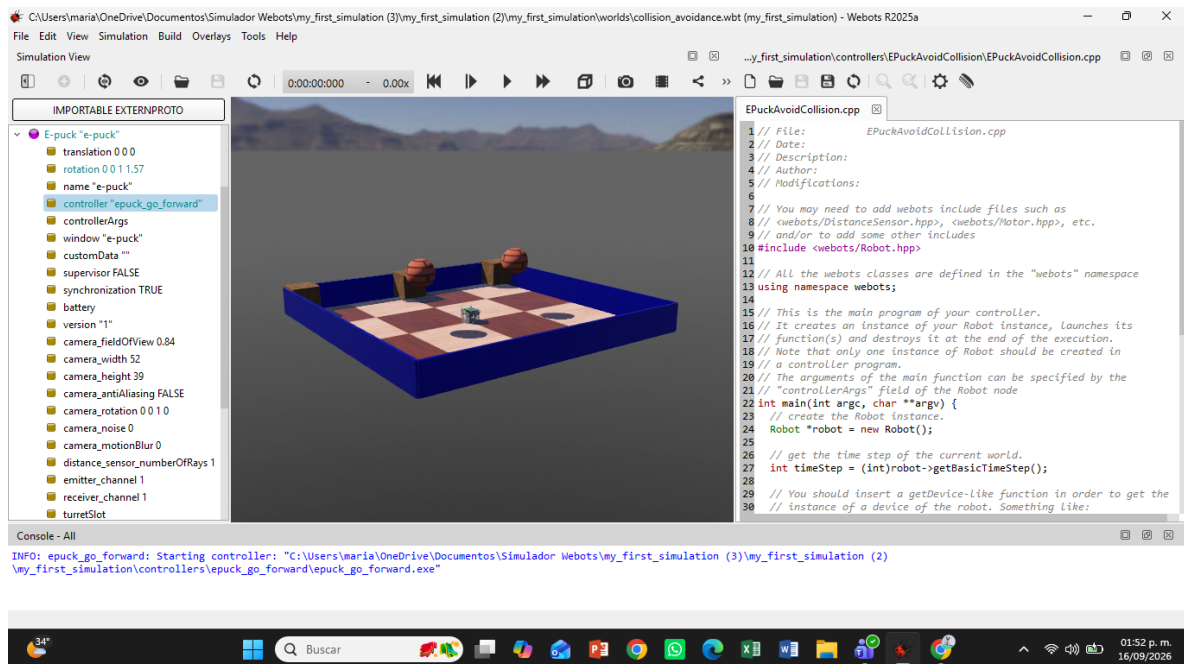
Componentes:

- **API del Controlador:** Interfaz de programación que otorga acceso a los sensores y actuadores simulados del robot. Se emplean librerías específicas como `webots/robot.h`, `webots/distance_sensor.h` y `webots/motor.h`.
- **Sensores de Distancia (DistanceSensor):** Los 8 sensores infrarrojos del *e-puck* (nombrados de `ps0` a `ps7`).
- **Motores y Actuadores (Motor):** Dispositivos que controlan las ruedas izquierda y derecha (*left wheel motor* y *right wheel motor*).
- **Paso de Simulación y TIME_STEP:** Macro que define la duración de cada paso de física en milisegundos (típicamente un múltiplo del `basicTimeStep` del nodo `WorldInfo`).

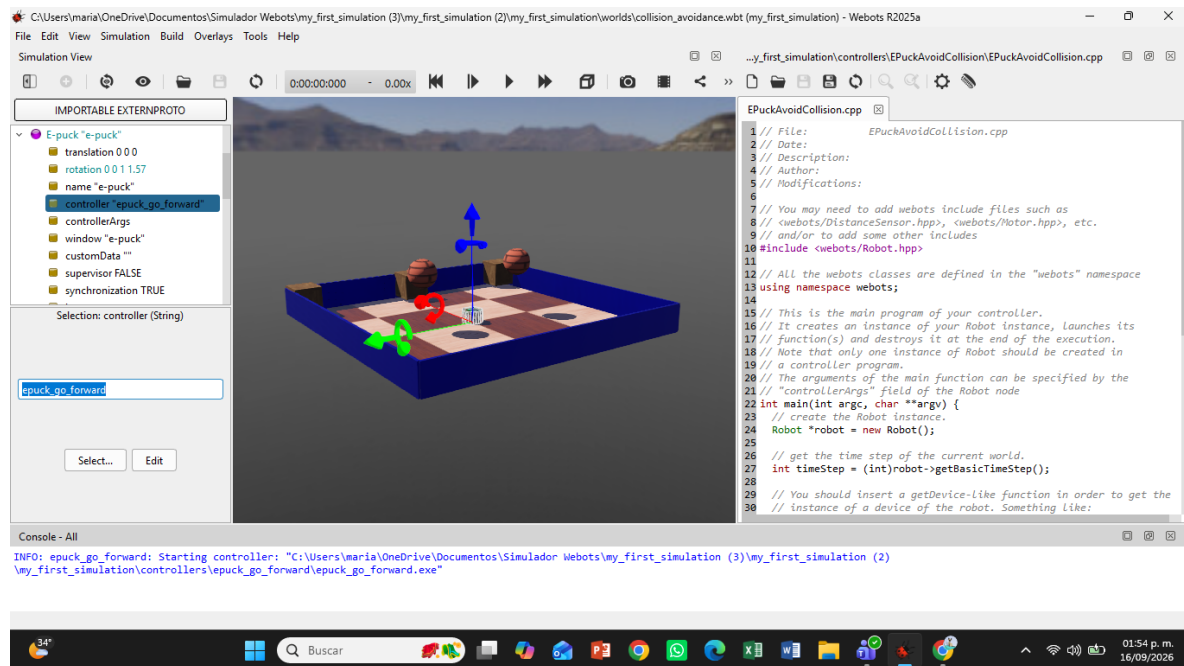
Se partió de la simulación previa guardando una copia del archivo de entorno bajo el nombre de `collision_avoidance.wbt`, se creó un nuevo controlador (por ejemplo, denominado `e-puck_avoid_collision`) utilizando el asistente de Webots, se modificó el campo `controller` del nodo del robot *E-puck* en el árbol de escena para asociarlo al nuevo controlador desarrollado.



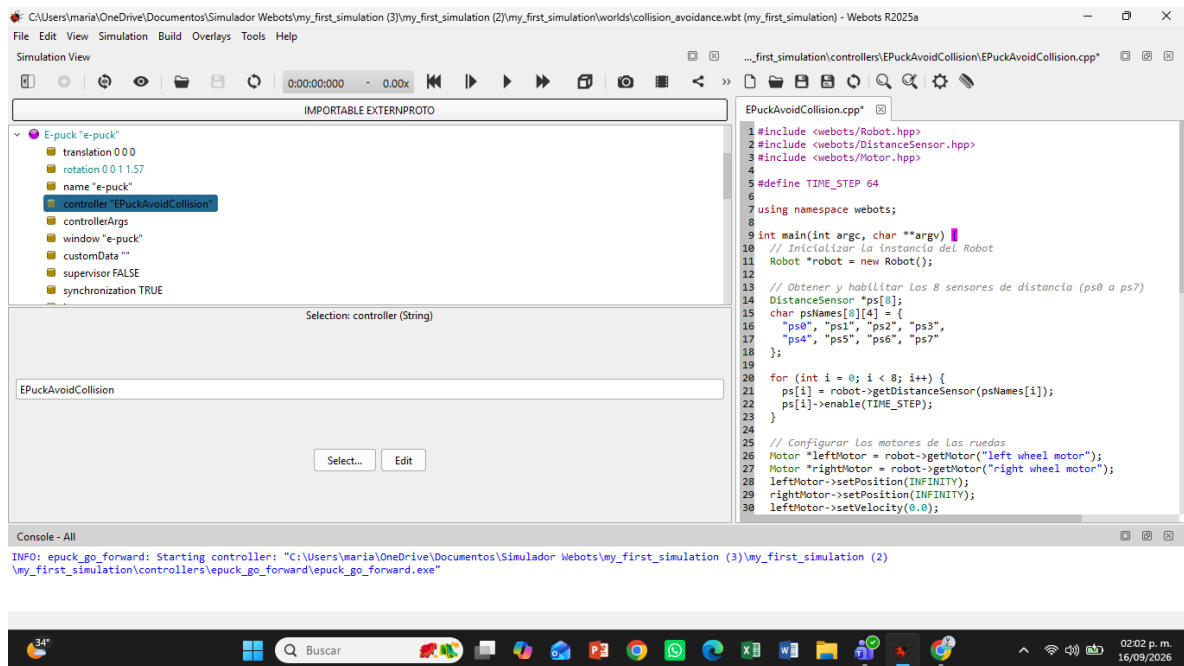
se muestra la plantilla básica en C++ (EPuckAvoidCollision.cpp) generada por el asistente de Webots, la cual incluye la inclusión de la librería principal (<webots/Robot.hpp>) y la estructura base de la función main con la inicialización del objeto Robot y la lectura del timeStep.



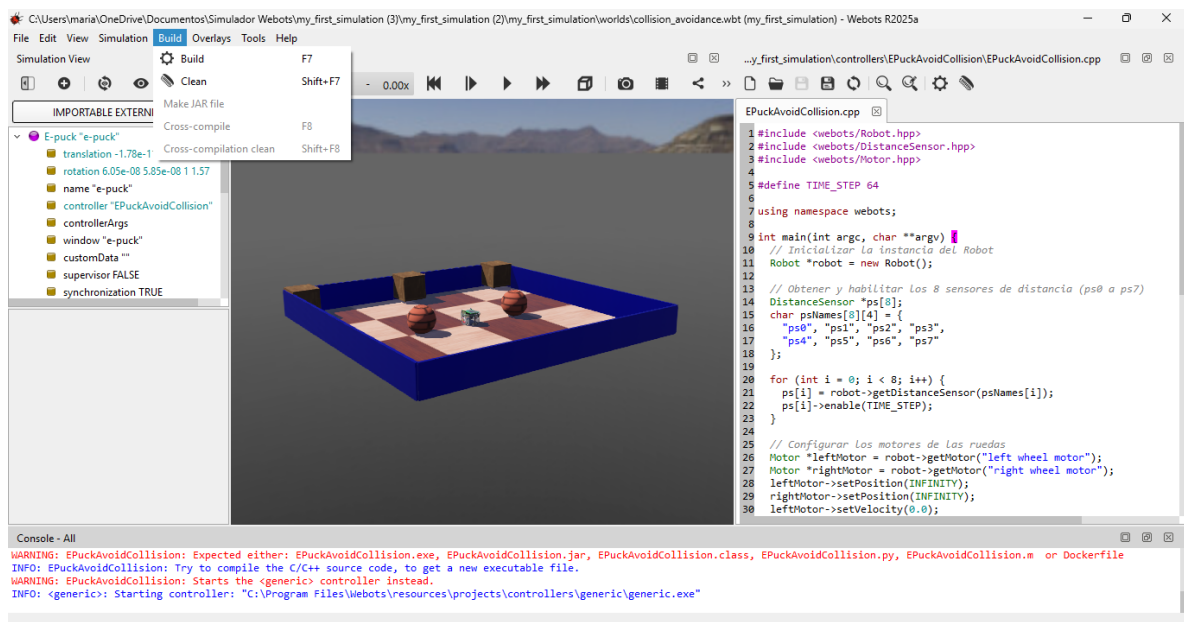
El panel izquierdo despliega todos los parámetros configurables del robot, destacando el campo controller, el cual indica el controlador que actualmente tiene asignado el robot en la simulación.



Se muestra que el robot seguía apuntando al controlador anterior (epuck_go_forward), por lo que es necesario actualizarlo para que ejecute el nuevo código de evasión de colisiones.

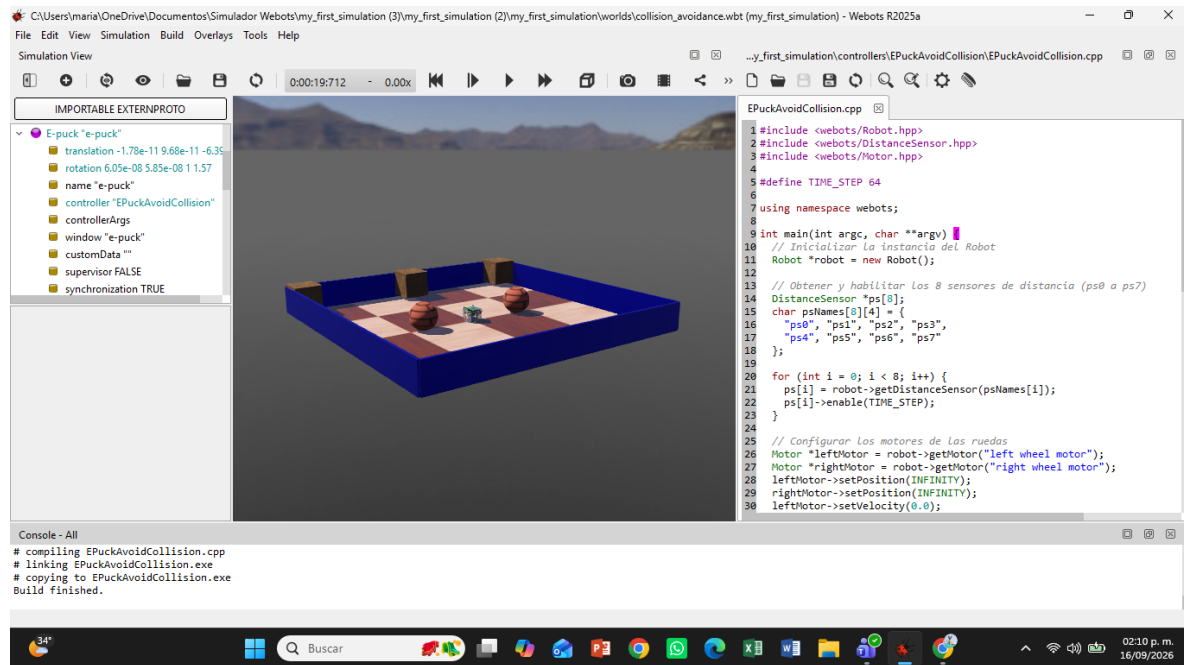


Se observa el bucle for encargado de inicializar de forma iterativa los 8 sensores infrarrojos (ps0 a ps7) utilizando la frecuencia definida por TIME_STEP, así como la configuración de los motores de las ruedas en modo de velocidad continua (INFINITY).

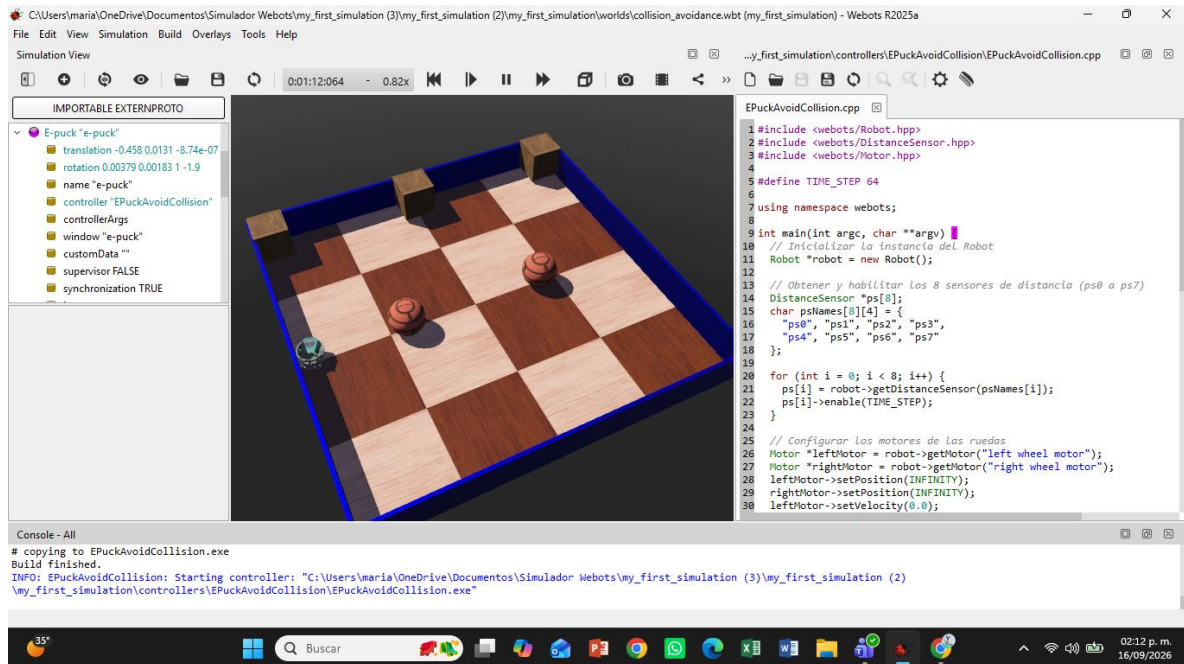


Al intentar ejecutar o correr la simulación sin haber compilado previamente el código fuente, la consola inferior arroja una advertencia (*Warning*). Webots indica que no

encuentra el archivo ejecutable compilado (.exe), ejecutando temporalmente un controlador genérico por defecto.



La consola muestra los mensajes de éxito (# compiling, # linking, y # copying to EPuckAvoidCollision.exe), confirmando que el código está listo para gobernar al robot de forma autónoma.



Le di play de nuevo a la simulación y ahora sí la consola me marcó que el controlador nuevo arrancó chido (Starting controller...). El robot empezó a moverse solo por el escenario y esquivaba los obstáculos usando sus sensores.